

# SOUPCON Build Log: FRTCON → SOUPCON

Shared for portfolio & demonstration purposes. All rights reserved.

This is the working log for turning a fork of `frtcon.com` into `soupcon.org` (“Soup Conditions”), and for everything built after launch. Each numbered item records what was done, why, and how it was checked. Code comments that say “SOUP\_PLAN.md item N” refer to these numbers.

This was a new app on top of FRTCON’s plumbing (React and Vite, the NWS API, caching, the PWA setup), not a code review. `frtcon.com` itself was left alone and still runs as its own site.

**Summary:** items 1-11 are the rebuild, 12-13 are hosting and launch, and 14-27 are fixes and features added after launch. The biggest of those are worldwide weather (22), city search (24), and the Sources panel and its chart (20, 26, 27).

---

## Starting decisions (2026-09-27)

- **Concept.** Keep FRTCON’s shape, 1 = worst and 5 = best, but rate rain instead of winter storms:

| Level    | Meaning                                                  |
|----------|----------------------------------------------------------|
| <b>1</b> | Currently raining                                        |
| <b>2</b> | Rain expected in the next 12 hours                       |
| <b>3</b> | Rain expected in the next 48 hours (but not the next 12) |
| <b>4</b> | Cloudy and damp, no rain expected soon                   |
| <b>5</b> | Clear and sunny for the next several days                |

- **Data.** The NWS hourly forecast drives the 12- and 48-hour windows, a nearby station’s latest observation drives “raining now,” and the multi-day forecast drives cloudy vs. clear. NWS alerts are not part of the score.
- **Recipe.** Start with one placeholder soup recipe. Multiple recipes come later (item 18), so no rotation system up front.
- **Hosting.** `soupcon.org` gets its own domain and site config, set up like `frtcon.com`’s but kept separate.

Questions left open at the start, each answered in the item that ran into it: whether to keep the alerts panel (item 3), what replaces the snow animation (item 5), zone name vs. city name as the location label (item 2), how to handle stations that haven’t reported recently (item 2), and the visual theme (item 7).

---

## The rebuild

### 1. Classification logic (`src/lib/soupcon.js`)

A new pure module, `classifySoupcon`, replaced FRTCON’s alert classifier. It works through the levels from 1 to 5 and the first match wins:

- **Level 1:** the station observation reports rain. With no observation, the current hourly forecast period is used instead.
- **Levels 2 and 3:** rain in the hourly forecast, split at the 12-hour mark.
- **Level 4:** the next four extended (12-hour) periods read cloudy or damp.
- **Level 5:** none of the above.

Rain is detected by keywords in `shortForecast` and `textDescription` (rain, showers, thunderstorm, drizzle, sprinkles), a 40%+ chance of precipitation, or a measured amount in the last hour.

I added vitest from the start (FRTCON never had tests). The first 19 tests covered each level, the 12-hour boundary, the no-observation fallback, and the four-period window.

Vite 8 needs Node 20.19+, and the dev machine's system Node was older, so the build and tests ran on a newer Node. (See "Lessons learned" in `CONTEXT.md`.)

## 2. API layer (`src/lib/weatherApi.js`)

Three new functions: `getHourlyForecast`, `getExtendedForecast` and `getCurrentConditions`. They share one cached `/points` lookup (`getGridpointInfo`), so a location costs one `/points` request instead of three.

- **Location label:** the city and state from the `/points` response ("Seattle, WA"), instead of FRTCON's forecast-zone name.
- **Stale stations:** try up to 5 nearby stations and use the first with a reading less than 90 minutes old. If none qualifies, return `null` and let the classifier use the hourly forecast.
- **Cache times:** gridpoint 1 hour, hourly forecast 30 minutes, extended forecast 2 hours, observations 5 minutes.

Checked against live `api.weather.gov` responses. I collected every distinct forecast phrase from 48 hours of data in five rain-prone cities (Miami, New Orleans, Houston, Orlando, Tampa) and confirmed the keyword lists caught all of them.

## 3. Wiring it into `App.jsx`

- **Alerts panel kept**, but separate from the score. Flood alerts still matter for a rain app. The "alerts driving the score" block was removed, since alerts no longer drive anything.
- The four data sources and alerts load in parallel. In practice they share one `/points` request, because concurrent calls for the same location are merged.
- The refresh logic (refresh on a timer, and when the tab becomes visible again) carried over from FRTCON and now reloads all four sources.
- The zone lookup and its helpers were deleted once nothing used them.

## 4. Content

- `soupMessages.js` replaced FRTCON's French-toast copy: a headline, title and 20 rotating commentary lines per level, all about rain and soup.
- A placeholder chicken noodle soup recipe replaced the French toast one.
- The page title, subtitle, menu item, share text and status messages were rewritten for SOUPCON.

## 5. Component renames

- `FrtconBadge` → `SoupconBadge` and `FrtconMessage` → `SoupconMessage`, with their hard-coded labels and footnote rewritten for the new concept.
- **Snow became rain.** `SnowOverlay` became `RainOverlay`: thin streaks that fall straight down instead of drifting flakes. FRTCON showed snow at every level. SOUPCON shows rain only at levels 1-3 (140, 90 and 40 drops) and none at 4-5, since rain on screen would contradict "no rain expected."
- The old `frtcon.js` was deleted.

## 6. CSS renames

Every `.frtcon-*` class and id became `.soupcon-*`. The "matching alerts" styles were deleted outright, since item 3 removed what they styled.

## 7. Branding

- **Icon.** I described a two-tone purple bowl under blue raindrops on lavender. The first version wasn't bold enough, so I edited it by hand in Inkscape (larger bowl, larger drops, richer colors), and every icon size was generated from that master SVG.
  - The Apple touch icon is full-bleed, since iOS rounds the corners itself.
  - The maskable Android icon is scaled to 85%. I worked out that the bold master's edges sat just outside the safe zone for a circular crop.
  - The favicon was checked at small sizes.
- **Palette.** The blue accents stayed, matching the raindrops, and the navy backgrounds and borders became purple at the same lightness. That gives a purple shell with blue rain accents, like the icon. Brand and meaning-carrying colors (Facebook blue, error red, the amber condition box, the five level colors) didn't change.
- `package.json`, `index.html` meta and Open Graph tags, and `manifest.json` were updated.

## 8. Domain and crawler files

`robots.txt` and `sitemap.xml` now point to `soupcon.org`. `index.md` (the markdown version of the page for Accept: text/markdown requests) was rewritten for SOUPCON, not just search-and-replaced. The Share button's URL was updated here too.

## 9. Storage keys

All `localStorage` prefixes went from `frtcon_` to `soupcon_`. The same went for the offline page's `sessionStorage` key and element ids in `sw.js` and `main.jsx`. `soupcon.org` is a new origin, so there were no old keys to migrate.

## 10. Docs

`README.md` and `CONTEXT.md` were rewritten for SOUPCON, and `FRTCON_PLAN.md` was marked as historical.

## 11. Tests

On top of item 1's tests, I added day/night wording ("Sunny" vs. "Clear", "Mostly Sunny" vs. "Mostly Clear"). Every real forecast phrase collected in item 2 became a permanent regression test. 22 tests.

---

## Hosting and launch

### 12. Hosting

I set this up myself: a new route on the existing Cloudflare Tunnel and a new Apache site serving from loopback, with a `dev/` copy beside production.

- **No origin TLS certificate.** The original plan assumed one. Reading the actual tunnel config showed the tunnel talks plain HTTP to Apache over loopback for every site, `frtcon.com` included, and Cloudflare handles HTTPS at the edge. A certificate would add upkeep for nothing.
- Checked live: `soupcon.org`, `www` and `/dev/` all return 200, and file permissions are correct.
- **Found and fixed:** `mod_headers` wasn't enabled, so the `.htaccess` Link headers were quietly dropped. `frtcon.com` had the same problem.
- **Found, still open:** Cloudflare's managed `robots.txt` adds its own rules ahead of the app's (see `CONTEXT.md`).

### 13. Launch

Live at the root and at /dev/, checked over HTTP. The PWA install flow on a real phone was still untested at this point.

---

## After launch

### 14. Rain vs. snow

*“What happens if it snows instead of rains?”*

Looking into it turned up a real bug. A plain “Snow” forecast matched no keyword list and fell all the way through to level 5, “Clear and sunny,” while it was snowing. “Snow Showers” was also miscounted as rain because of the word “showers.”

- Added snow keywords (snow, sleet, blizzard, flurries, wintry mix), checked before the rain keywords. Freezing rain stays rain.
- The result now carries precipType (rain or snow), which picks the wording (“It’s snowing right now”) and the animation. A new SnowOverlay brought back FRTCON’s drifting flakes alongside the rain streaks.
- The rotating commentary stays rain-themed. A full snow set (about 60 more lines) was considered and left for later.
- Added a regression test for the “Snow → clear and sunny” bug. 30 tests.

### 15. FRTCON link when snow is coming

A “Snow soon, check your FRTCON!” pill next to the Share button links to frtcon.com whenever snow drives the forecast. Since FRTCON is US-only, the pill was later hidden for non-US locations.

### 16. Post-launch bug review

A full read of the code plus live probes of NWS data across seven cities found two clear bugs:

- **Forecast periods that had already ended counted as current.** Live hourly data often starts with the hour that just ended, so that hour could read as “raining now.” Ended periods are now dropped.
- **Blank observations counted as “not raining.”** Some stations, including Chicago’s nearest, post fresh readings with no description. Those are now skipped, so the next station or the hourly fallback is used.

It also raised four judgment calls, which I decided:

- “Partly Cloudy” (night wording) and “Partly Sunny” (day wording) are the same sky, so neither counts as cloudy. Before this, every night’s “Partly Cloudy” kept dragging fair outlooks down to level 4.
- “Slight Chance” no longer means “raining right now.” The level-1 hourly fallback needs 40%+, or unhedged wording.
- Rain anywhere in the next four extended periods means level 4, never level 5.
- “In Vicinity” station readings still count as raining now.

Smaller fixes from the same review:

- Station lookups now run in parallel. One at a time, with a 10-second timeout each, a slow NWS could stall a lookup for about a minute.
- An aborted /points request is no longer reused by a later lookup.
- An empty hourly forecast is an error instead of “clear and sunny.”
- “Ice pellets” and “hail” are now recognized.

The new API tests were confirmed to fail on the old code and pass on the new. 45 tests.

## 17. Station radius and snow pill wording

*“Why 5 stations? Can we use stations within 60 miles, with at least 2?”*

- `selectObservationStations` sorts stations by real distance (haversine), since NWS’s order is only roughly right. It takes all within 60 miles, but at least 2 and at most 5. The cap stays because dense areas have dozens of stations in range (41 near Seattle).
- Checked live for crowded and remote places (Seattle, Chicago, Ely NV, Big Bend TX, Jordan MT).
- The snow pill reads “Snow! Check FRTCON!” when it’s already snowing.
- **First real browser test of the rebuild.** I ran a headless Chrome against the production build and checked ZIP and location lookups, the rain overlay, alerts, saving and auto-resuming the last lookup, error handling, the recipe modal’s keyboard focus, Share, simulated snow (rewritten API responses), and phone width.
- **Fixed along the way:** the recipe modal had been showing in the browser’s default serif font (a leftover from FRTCON).

## 18. Real recipes

The recipes come from our family recipe wiki. Recipes in its SOUPCON category are pulled from the wiki’s raw source and saved as static files in the app, one file per recipe in `src/data/recipes/`. Nothing loads from the wiki at run time, so a wiki edit can’t break the app.

- Each recipe gets its own menu item and reuses the existing `RecipeModal` and its print view.
- Recipes can include their story section (“Soup Tales!”) when the wiki page has one.
- **Pulled from the raw source, not summarized.** An AI web-fetch tool rewrote ingredient text and even made up a title, so the extraction reads the wiki’s exact source instead.
- Problems in the source text (a leftover “cilantro” after the ingredient became parsley, and a stray “mirin” step) were copied as written, to be fixed on the wiki itself.
- Ingredient groups (“Potsticker Sauce:”, “Garnish:”) are plain list entries, shown in bold.
- Current recipes: Potsticker Soup, Senegalese Chicken Soup, Lasagna Soup.

## 19. Soup of the day

*“Pick a soup of the day... cache it until midnight local time.”*

- `soupOfTheDay.js` stores the pick with the local calendar date, not a timeout, so it changes exactly at local midnight.
- If the saved recipe no longer exists, or `localStorage` is unavailable, it just picks again.
- The pill reads “Lets make {recipe}!” in amber, next to the Share button, and opens that recipe.
- Later fix: a tab or installed app left open overnight kept yesterday’s soup. The app now re-checks at local midnight (calculated from the calendar, so daylight-saving changes don’t shift it) and whenever the page becomes visible again.

## 20. Sources panel

*ZIP 12577 showed level 1 with nothing on radar.*

The nearest station was blank, and the next one had one glitchy “Light Rain” reading between “Cloudy” ones. Instead of making the classifier second-guess stations, I chose to make the data visible: a collapsible **Sources** panel under the alerts. It shows each nearby station’s reading (used or skipped, and why) plus the forecast periods the classifier read. Station readings load only while the panel is open.

The classifier still trusts the nearest usable station. Requiring a second station or forecast agreement was discussed and not done.

## 21. lat/lon URL parameters

?lat=...&lon=... loads a specific location. Both values must be present and in range. They take priority over the remembered lookup but are never saved as the visitor's own lookup method. A small note shows the coordinates being used.

## 22. Worldwide weather (Open-Meteo)

*"Is there a world weather service we can use outside the US?"*

NWS only covers the US. I chose Open-Meteo as the fallback: no API key, open CORS, worldwide coverage. Alternatives: MET Norway requires an identifying User-Agent (the same problem as NWS), and the others need an API key that would be exposed in the browser. Open-Meteo's free tier is non-commercial, which fits for now.

Built in steps:

1. **Wording:** the ZIP button became "Search US ZIP."
2. **API layer** (`openMeteoApi.js`): one request returns current, hourly and daily data. Before building it, I checked live responses for 11 cities: field names, CORS, errors, and time handling. That led to Unix timestamps and a 48-hour series starting at the current hour.
3. **Classifier** (`soupconOpenMeteo.js`, `wmoCodes.js`): a separate code path using WMO weather codes, sharing one result builder with the NWS classifier.
  - For levels 4 and 5, the 48 hours are split into four 12-hour blocks. A block is cloudy if it has an overcast or fog hour or averages 70%+ cloud cover.
  - Rain is decided by weather code alone. An 80% chance under an "Overcast" code (seen in Reykjavik) shouldn't count as rain.
4. **Raining now** uses Open-Meteo's current block directly: measured snow, then rain, then the weather code. It's a model estimate, so one bad station reading can't cause level 1.
5. **Place names** (`reverseGeocode.js`): BigDataCloud's free reverse geocoder. Its terms allow only the device's own location, so it runs only for browser-geolocation lookups. URL coordinates show as "Lat 51.51, Lon -0.13." A failure never blocks the forecast.
6. **Routing** (`weatherProvider.js`): ask NWS first and switch to Open-Meteo only on a 404 from /points. London and Vancouver return 404, while Puerto Rico, Hawaii, Alaska and Guam work. An NWS outage is still an error, not a silent switch. A refresh stays with the same provider.
7. **Alerts:** Open-Meteo has none, so the alerts section is hidden outside the US.
8. **Sources panel:** for Open-Meteo it shows the model's current values, hourly rows, the four cloud blocks (and why each is cloudy or not), and the daily outlook. Daily rows now use the location's own time zone. With UTC, Tokyo's days were shifted.
9. **Refresh:** handled in step 6.
10. **Tests:** WMO mapping, routing (404 switches providers, 500 doesn't), place names, normalization, and every level for rain and snow. 126 tests.
11. **Docs and browser check:** checked in headless Chrome with real APIs for Seattle, London, Mumbai and an emulated London location, plus phone width. **Found:** non-US lookups were sending a doomed NWS alerts request. Alerts now wait until NWS confirms the point is in coverage.

Not built: non-US postal codes and non-US alerts. The snow codes are tested only, since nothing was forecasting snow at the time.

## 23. Clearer location errors

*A friend in Romania saw "Timeout expired. Try entering a US ZIP code instead."*

That message is the browser's own, and a non-US visitor can't use a ZIP. `geolocationError.js` now gives a plain explanation for each failure (timed out after 45 seconds, position unavailable, permission denied), followed by a hint that fits everyone.

I considered dropping the high-accuracy retry so failures would report sooner, and decided to keep it. Low accuracy doesn't always work, and the retry is what rescues those cases.

## 24. Country dropdown and city search

A **Country** dropdown sits next to the ZIP field, set to USA by default. Picking any other country swaps in a city field and a “Search city” button.

- `placeSearch.js` uses Open-Meteo’s geocoding search, limited to the chosen country. It returns populated places only, so “Cluj” finds the city and not its airport. One match goes straight to the forecast, and several show a “Which one?” list.
- Country names come from the browser (`Intl.DisplayNames`), from a hand-written list of country codes.
- The chosen place is remembered and resumed like a ZIP. A country NWS covers (Puerto Rico) still uses NWS.
- Checked in headless Chrome: Romania/Cluj, Puerto Rico/San Juan, not-found, switching back to USA, resume after reload, and phone width.

## 25. Review of items 19-24

A full pass over the plan against the code found these:

- A stale “Which one?” list could stay on screen after switching to browser location, or land under the wrong country.
- Some wrong wording in the Sources panel.
- Docs that had drifted from the code.
- The soup of the day not changing at midnight (fixed in item 19).
- An unused `skipCache` option, now removed.

## 26. “Decided by” line in Sources

*ZIP 12577 showed level 3 while every row in Sources looked dry.*

The level was right. The rain was 47 hours out, past the rows the panel listed. Each classifier now records the one reading that set the level, and Sources leads with it, for example: “SOUPCON3 decided by: hourly forecast Fri 2:00 PM - Chance Rain Showers, 38%.” Because the classifier records it, the panel can’t show a different answer. Checked against live NWS data for 12577.

## 27. 48-hour chart in Sources

The 12-row hourly list became a 48-hour chart: rain chance as a filled area, cloud cover as a line, and a dashed marker on the deciding hour.

- I picked **uPlot** for its small size, over Chart.js, Recharts and visx. It loads (about 23 KB gzipped) only when Sources is opened.
- US cloud cover comes from NWS’s raw gridpoint `skyCover` percentage, so there’s no guessing from words. It’s fetched only while Sources is open and isn’t used for scoring.
- Chart colors were checked for contrast against the card background. A pale lavender for clouds read as gray and was replaced.
- The full hourly data is in a collapsed “Hourly details” list under the chart.
- Checked in headless Chrome at desktop and phone widths, for the US and London.

---

## Still open

- Check a real snow forecast on the Open-Meteo path when one comes along.
- Maybe show place names for `?lat=&lon=` links without the reverse geocoder.
- Test the PWA install on a real phone, and test recipe printing.
- Decide what to do about Cloudflare’s managed robots.txt.
- Settled, staying as they are: an untyped precipitation reading counts as rain; the 90-minute station limit; NWS’s hard line breaks in alert text.

---

© Jeffrey Morton. Shared for portfolio & demonstration purposes. All rights reserved.